

1. Introduction

The Shop Billing System is a console-based Python application that automates billing for a small shop. Manual billing is slow and error-prone, since totals must be calculated by hand and records are hard to keep. This program takes item details from the user, calculates the total, prints a formatted bill, and saves a copy as a text file.

2. Objectives

- Replace manual bill calculation with an automated program.
- Validate user input so wrong values (letters, negative prices) don't break the program.
- Display a clear, readable bill on screen.
- Store every bill as a timestamped file for future reference.

3. Scope

The system handles one bill per run for any number of items. It is meant for small shops and for learning core Python concepts. It doesn't include a database, GUI, tax or discount handling, or inventory management.

4. Technology and Requirements

- **Language:** Python 3
- **Modules used:** OS (folder creation and file paths) and `datetime` (date and time stamps)
- **Hardware:** Any computer that can run Python
- **Software:** Python 3.x interpreter and a terminal or IDE (IDLE, VS Code, PyCharm)

5. System Design

5.1 Functions

Function	Purpose
<code>get_int(text)</code>	Repeatedly asks until the user enters a valid whole number (used for item count and quantity).
<code>get_float(text)</code>	Repeatedly asks until the user enters a valid non-negative decimal number (used for price).
<code>main()</code>	Controls the whole flow: input, calculation, display, and file saving.

5.2 Data Storage

Four parallel lists hold the data for each item: `names`, `qty`, `price`, and `totals`. The same index in every list refers to the same item.

5.3 Workflow

- 1 The program prints the heading and asks for the number of items.
- 2 If the count is 0, it prints "Nothing to bill" and exits.
- 3 For each item, it asks for name, quantity, and price. An empty name becomes Item_1, Item_2, and so on.
- 4 The item total is calculated as quantity x price.
- 5 The bill is printed with each item's details and the grand total.
- 6 A `bills` folder is created if it doesn't exist, and the bill is saved as `bill_YYYYMMDD_HHMMSS.txt`.

6. Implementation Details

Input validation. `get_int` uses `isdigit()` to reject letters, symbols, and negative numbers. `get_float` uses `try/except` to catch invalid input and rejects negative values.

Default naming. If the user leaves the name blank, an automatic name is assigned so the bill never has an empty entry.

Calculation. Each item's total is $q * p$, and the grand total is accumulated in a loop.

Formatting. `ljust(15)` left-aligns item names in a 15-character column so the bill looks tidy.

File handling. `os.makedirs(folder, exist_ok=True)` creates the `bills` folder without error if it already exists. Using the current date and time in the filename means each bill gets a unique name and old bills are never overwritten. Files are written with UTF-8 encoding.

7. Sample Output

```

===== SHOP BILLING =====
How many items? : 2

Item 1
Name: Rice
Qty: 2
Price: 60

Item 2
Name: Sugar
Qty: 1
Price: 45.5

----- BILL -----
Name          Qty   Price   Total
Rice           2     60.0    120.0
Sugar          1     45.5     45.5
-----
Grand Total = 165.5
-----
Bill saved: bill_20260929_103015.txt

```

8. Testing

Test Case	Input	Expected Result
Valid bill	2 items with valid data	Bill printed and saved
Zero items	0	"Nothing to bill." and exit
Non-numeric count	abc	"Enter proper number please", asks again
Negative price	-5	"Amount can't be negative", asks again
Invalid price	abc	"Invalid amount", asks again
Blank item name	Press Enter	Name becomes Item_1
Multiple runs	Run twice	Two separate files in bills/

9. Advantages

- Simple, easy-to-understand code, suitable for beginners.
- Reliable input validation.
- Automatic, permanent record of every bill.
- No external libraries required.

10. Limitations

- Only whole-number quantities are accepted, so items sold by weight (e.g. 1.5 kg) aren't supported.

- No tax (GST), discount, or customer details.
- The bill columns can misalign when prices or totals have different lengths.
- Prices show floating-point results like 45.5 rather than a fixed two decimal places.
- The file is opened with `open()/close()`, so it may stay open if an error occurs while writing.
- Only one bill can be created per program run.
- It is command-line only, with no graphical interface.

11. Future Enhancements

- Add GST and discount calculation.
- Support decimal quantities for items sold by weight.
- Format money to two decimals (e.g. `f"{x:.2f}"`).
- Use a `with open( . . . )` block for safer file handling.
- Store data in a dictionary or list of objects instead of parallel lists.
- Add a database (SQLite) for sales history and reports.
- Build a GUI with Tkinter, or a web version.
- Add PDF bill generation and printing.
- Add a product catalogue with stored prices and stock tracking.

12. Conclusion

The Shop Billing System meets its goal of automating basic billing with validated input, a formatted on-screen bill, and saved receipts. It demonstrates core Python concepts: functions, loops, lists, string formatting, exception handling, and file handling. With the suggested enhancements it could grow into a fuller point-of-sale application.